

---

# Aplicație de gestionat timpul de studiu

Documentație de proiect

Metode de Dezvoltare Software

---

## **Echipa de proiect**

Florea Bogdan  
Cojocaru Gabriele  
Zamfir Radu-Adrian

---

## Cuprins

---

|                                                   |   |
|---------------------------------------------------|---|
| Cuprins .....                                     | 2 |
| 1. Introducere.....                               | 3 |
| 1.1 Obiectivele proiectului.....                  | 3 |
| 2. Tehnologii utilizate .....                     | 3 |
| 3. Arhitectura aplicației.....                    | 4 |
| 3.1 Componente principale.....                    | 4 |
| 4. Structura bazei de date .....                  | 4 |
| 4.1 Tabela UTILIZATORI.....                       | 4 |
| 4.2 Tabela SESIUNI_STUDIU .....                   | 5 |
| 4.3 Tabela COMPETENTE.....                        | 5 |
| 5. Funcționalități.....                           | 5 |
| 5.1 Gestiunea conturilor .....                    | 5 |
| 5.2 Componentele de inteligență artificială ..... | 6 |
| 5.2.1 Generarea programului de studiu.....        | 6 |
| 5.2.2 Estimarea timpului de învățare .....        | 6 |
| 5.3 Gestiunea competențelor.....                  | 6 |
| 6. Interfața de programare (API REST) .....       | 7 |
| 6.1 Conturi de utilizator.....                    | 7 |
| 6.2 Studiu și competențe .....                    | 7 |
| 6.3 Componente AI.....                            | 7 |
| 7. Decizii de proiectare.....                     | 7 |
| 7.1 Inteligența artificială pe server .....       | 7 |
| 7.2 Reîncercare automată la suprasolicitare.....  | 8 |
| 7.3 Conexiune robustă la baza de date .....       | 8 |
| 7.4 Răspuns AI în format structurat .....         | 8 |
| 8. Testare.....                                   | 8 |
| 9. Limitări și dezvoltări viitoare .....          | 9 |
| 10. Concluzii .....                               | 9 |

# 1. Introducere

Aplicația de gestionat timpul de studiu este un instrument care ajută utilizatorii să își organizeze mai eficient timpul petrecut învățând, contribuind atât la un echilibru sănătos între muncă și viața personală, cât și la dobândirea de noi competențe utile în diverse situații.

Elementul central al aplicației îl reprezintă două componente bazate pe inteligență artificială. Prima generează automat, pe baza activității de studiu din ultima săptămână, un program de studiu pentru săptămâna următoare. Acest program poate fi vizualizat de utilizator în interfață. A doua componentă, pornind de la datele introduse de utilizator, estimează timpul necesar pentru a învăța o anumită competență.

Utilizatorul își poate gestiona contul, își poate înregistra competențele pe care dorește să le dobândească și poate urmări progresul prin compararea orelor studiate cu orele estimate de sistem.

## 1.1 Obiectivele proiectului

- Oferirea unui instrument simplu pentru planificarea și urmărirea timpului de studiu.
- Integrarea a două componente de inteligență artificială: generarea programului săptămânal și estimarea timpului de învățare.
- Gestiunea completă a conturilor de utilizator (creare, autentificare, modificare, ștergere).
- Urmărirea progresului pe competențe, prin raportul ore studiate / ore estimate.
- 

## 2. Tehnologii utilizate

Proiectul a fost dezvoltat folosind limbajele JavaScript și SQL, conform cerinței inițiale, organizate într-o arhitectură client-server.

| Componentă               | Tehnologie            | Rol                                                             |
|--------------------------|-----------------------|-----------------------------------------------------------------|
| Interfață (client)       | Java Swing            | Aplicație desktop cu interfață grafică, organizată pe tab-uri.  |
| Server (backend)         | Node.js + Express     | API REST care expune funcționalitățile aplicației.              |
| Bază de date             | PostgreSQL (Supabase) | Stocarea utilizatorilor, sesiunilor de studiu și competențelor. |
| Inteligență artificială  | Google Gemini API     | Generarea programului și estimarea timpului de învățare.        |
| Comunicare client-server | HTTP / JSON           | Schimb de date între aplicația Java și server.                  |
| Prelucrare JSON în Java  | Gson                  | Construirea și parsarea mesajelor JSON.                         |

### 3. Arhitectura aplicației

Aplicația este structurată pe trei niveluri. Interfața Java comunică exclusiv cu serverul Node.js prin cereri HTTP. Serverul este singurul care interacționează direct cu baza de date și cu serviciul de inteligență artificială. Acest model menține cheile de acces și logica sensibilă pe server, nu în aplicația distribuită utilizatorilor.

#### Fluxul de date:

```
Interfață Java (Swing) --HTTP/JSON--> Server Node.js (Express)
                                     |--> Bază de date PostgreSQL (Supabase)
                                     |--> Google Gemini API
```

Toate apelurile de rețea din interfața Java sunt executate pe un fir de execuție separat, prin intermediul clasei `SwingWorker`, pentru a nu bloca interfața grafică în timp ce se așteaptă răspunsul serverului. Apelurile către serviciul de inteligență artificială pot dura câteva secunde, motiv pentru care această abordare este esențială pentru o experiență fluidă.

#### 3.1 Componente principale

- **MDSUI** – clasa principală a interfeței, care conține tab-urile și logica butoanelor.
- **ApiService** – clasă utilitară care centralizează toate apelurile HTTP către server.
- **Session** – clasă care reține utilizatorul autentificat pe durata rulării aplicației.
- **server.js** – serverul Node.js care definește rutele API și se conectează la baza de date și la Gemini.

### 4. Structura bazei de date

Baza de date conține trei tabele principale, găzduite pe platforma Supabase.

#### 4.1 Tabela UTILIZATORI

Stochează conturile utilizatorilor și preferințele lor de studiu.

| Coloană            | Descriere                                                                    |
|--------------------|------------------------------------------------------------------------------|
| ID                 | Identificator unic al utilizatorului (cheie primară).                        |
| NUME               | Numele de utilizator.                                                        |
| EMAIL              | Adresa de email, folosită la autentificare.                                  |
| PAROLA             | Parola contului.                                                             |
| NIVEL_CUNOSTINTE   | Nivelul general de cunoștințe (1 – începător, 2 – intermediar, 3 – avansat). |
| ORE_DISPONIBILE_ZI | Numărul de ore pe care utilizatorul le poate dedica studiului zilnic.        |

## 4.2 Tabela SESIUNI\_STUDIU

Înregistrează sesiunile de studiu efectuate, folosite de agentul de planificare pentru a genera programul.

| Coloană       | Descriere                                    |
|---------------|----------------------------------------------|
| USER_ID       | Referință către utilizatorul care a studiat. |
| DATA_SESIUNE  | Data sesiunii de studiu.                     |
| MATERIE       | Materia studiată.                            |
| DURATA_MINUTE | Durata sesiunii, exprimată în minute.        |

## 4.3 Tabela COMPETENTE

Reține competențele pe care utilizatorul dorește să le dobândească și progresul aferent.

| Coloană      | Descriere                                                              |
|--------------|------------------------------------------------------------------------|
| ID           | Identificator unic al competenței.                                     |
| USER_ID      | Referință către utilizatorul căruia îi aparține competența.            |
| DENUMIRE     | Denumirea competenței (de exemplu, „Algebra”).                         |
| ORE_ESTIMATE | Numărul de ore estimat de componenta AI pentru dobândirea competenței. |
| ORE_STUDIATE | Numărul de ore studiate efectiv, actualizat pe parcurs.                |

# 5. Funcționalități

## 5.1 Gestiunea conturilor

Tab-ul Account conține toate operațiile legate de contul utilizatorului. Datele de autentificare se introduc în câmpurile dedicate, iar fiecare buton declanșează o operație asupra serverului.

- **Create Account** – creează un cont nou cu numele, email-ul, parola, nivelul de cunoștințe și orele disponibile.
- **Log In** – verifică email-ul și parola; la succes, utilizatorul este reținut în sesiune.
- **Log Out** – golește sesiunea curentă și curăță câmpurile interfeței.
- **Change Account** – actualizează datele contului autentificat (nume, email, nivel, ore).
- **Change Password** – schimbă parola, după verificarea parolei actuale și confirmarea celei noi.
- **Delete Account** – șterge contul și datele asociate, după confirmarea email-ului și a parolei.

---

Operația de ștergere a contului folosește o tranzacție în baza de date: sunt eliminate mai întâi sesiunile și competențele asociate, apoi utilizatorul, garantând astfel că datele rămân consistente chiar dacă una dintre operații eșuează.

## 5.2 Componentele de inteligență artificială

Aplicația integrează două componente bazate pe modelul Google Gemini. Ambele rulează pe server, care construiește instrucțiunea (prompt-ul), apelează modelul și returnează rezultatul interfeței.

### 5.2.1 Generarea programului de studiu

Accesibil din tab-ul Schedule prin butonul „Genereaza program”. Serverul citește datele utilizatorului și sesiunile de studiu din ultima săptămână, apoi cere modelului să genereze un program echilibrat pentru zilele Luni–Duminică, respectând numărul de ore disponibile pe zi. Răspunsul este returnat în format JSON structurat și afișat într-un tabel pe zile.

### 5.2.2 Estimarea timpului de învățare

Accesibil din tab-ul Competente prin butonul „Estimeaza”. Pe baza denumirii competenței și a nivelului de cunoștințe al utilizatorului, modelul estimează numărul de ore necesare și oferă o scurtă justificare. Valoarea estimată este reținută și poate fi salvată ulterior ca parte a unei noi competențe.

## 5.3 Gestiunea competențelor

Tab-ul Competente permite utilizatorului să adauge competențe și să își urmărească progresul.

- **Estimeaza** – obține de la componenta AI numărul estimat de ore pentru competența introdusă.
- **Adauga competenta** – salvează competența în baza de date, folosind ca ore estimate valoarea obținută anterior.
- **Am studiat** – adaugă un număr de ore studiate la competența selectată din listă, actualizând progresul.

Competențele sunt afișate într-o listă, fiecare element având forma „Denumire: ore studiate / ore estimate”. Progresul este actualizat în timp real după fiecare sesiune de studiu înregistrată prin butonul „Am studiat”.

---

## 6. Interfața de programare (API REST)

---

Serverul Node.js expune următoarele rute, grupate pe categorii. Toate schimburile de date se realizează în format JSON.

### 6.1 Conturi de utilizator

| Metodă & rută                   | Descriere                                                  |
|---------------------------------|------------------------------------------------------------|
| GET /api/utilizatori            | Returnează lista utilizatorilor.                           |
| POST /api/utilizatori           | Creează un cont nou.                                       |
| POST /api/login                 | Autentifică un utilizator pe baza email-ului și a parolei. |
| PUT /api/utilizatori/:id        | Actualizează datele contului.                              |
| PUT /api/utilizatori/:id/parola | Schimbă parola, cu verificarea parolei actuale.            |
| DELETE /api/utilizatori         | Șterge contul și datele asociate.                          |

### 6.2 Studiu și competențe

| Metodă & rută                  | Descriere                                |
|--------------------------------|------------------------------------------|
| POST /api/sesiuni              | Înregistrează o sesiune de studiu.       |
| POST /api/competente           | Adaugă o competență nouă.                |
| GET /api/competente/:user_id   | Returnează competențele unui utilizator. |
| PUT /api/competente/:id/studiu | Adaugă ore studiate la o competență.     |

### 6.3 Componente AI

| Metodă & rută                       | Descriere                                     |
|-------------------------------------|-----------------------------------------------|
| POST /api/agent/program             | Generează programul de studiu săptămânal.     |
| POST /api/agent/estimare-competenta | Estimează timpul necesar pentru o competență. |

---

## 7. Decizii de proiectare

---

### 7.1 Inteligența artificială pe server

Apelurile către modelul Gemini sunt efectuate exclusiv din server, niciodată direct din aplicația Java. Cheia de acces la serviciul AI este păstrată într-un fișier de configurare pe server și nu este distribuită odată cu aplicația, evitând astfel expunerea ei.

---

## 7.2 Reîncercare automată la suprasolicitare

Serviciul AI poate returna ocazional o eroare temporară de suprasolicitare. Pentru a trata aceste situații, serverul folosește un mecanism de reîncercare cu pauze crescătoare între încercări. Astfel, dacă modelul este momentan indisponibil, cererea este reîncercată automat, fără ca utilizatorul să fie afectat.

## 7.3 Conexiune robustă la baza de date

Conexiunea la baza de date este configurată prin parametri separați (gazdă, port, utilizator, parolă, nume bază de date), nu printr-un singur șir de conexiune. Această abordare evită problemele de interpretare a caracterelor speciale din parolă și face configurarea mai sigură.

## 7.4 Răspuns AI în format structurat

Componentele AI sunt configurate să returneze răspunsuri în format JSON structurat. Acest lucru permite interfeței Java să parseze direct rezultatele și să le afișeze în tabele sau liste, fără prelucrări suplimentare ale textului.

---

# 8. Testare

Funcționalitățile au fost testate în etape, pentru a izola eventualele probleme:

1. Verificarea pornirii serverului și a conexiunii la baza de date, prin accesarea rutei de listare a utilizatorilor.
2. Pregătirea datelor de test (utilizatori și sesiuni de studiu) folosind un client REST.
3. Testarea directă a rutelor API, inclusiv a componentelor AI, înainte de integrarea în interfață.
4. Testarea fluxului complet prin interfața grafică: autentificare, generarea programului, estimarea și adăugarea competențelor, înregistrarea orelor studiate.

Această abordare pe niveluri a permis identificarea rapidă a sursei fiecărei probleme — server, bază de date, serviciu AI sau interfață.



---

## 9. Limitări și dezvoltări viitoare

---

- **Securitatea parolelor** – în versiunea actuală, parolele sunt stocate în text simplu. O îmbunătățire viitoare este aplicarea unei funcții de criptare (hashing) la salvarea parolelor.
- **Legarea sesiunilor de competențe** – în prezent, orele studiate se adaugă manual. O dezvoltare ulterioară ar fi corelarea automată a sesiunilor de studiu cu competențele corespunzătoare.
- **Editarea programului generat** – programul săptămânal poate fi extins pentru a permite utilizatorului să adauge sau să elimine manual perioade de studiu.
- **Indicator vizual de progres** – progresul pe competențe ar putea fi afișat printr-o bară de progres sau un procent, pentru o vizualizare mai clară.
- 

## 10. Concluzii

---

Proiectul demonstrează o aplicație funcțională de gestionare a timpului de studiu, construită pe o arhitectură client-server clară. Interfața Java comunică cu un server Node.js care integrează o bază de date PostgreSQL și două componente de inteligență artificială.

Aplicația acoperă întregul ciclu de utilizare: de la crearea și gestionarea contului, la generarea automată a programului de studiu, estimarea timpului necesar pentru noi competențe și urmărirea progresului. Structura modulară și separarea responsabilităților între client și server oferă o bază solidă pentru dezvoltările viitoare propuse.